

CCP2

Gestion des missions et des candidatures

Nom du projet : volunteer-platform

Projet backend Node.js / SQL

Objectif du projet

Plateforme de mise en relation entre bénévoles et associations

Permettre aux associations de publier des missions

Permettre aux bénévoles de candidater aux missions

Gérer les candidatures et leur statut

Sécuriser les accès selon les rôles

Stack technique

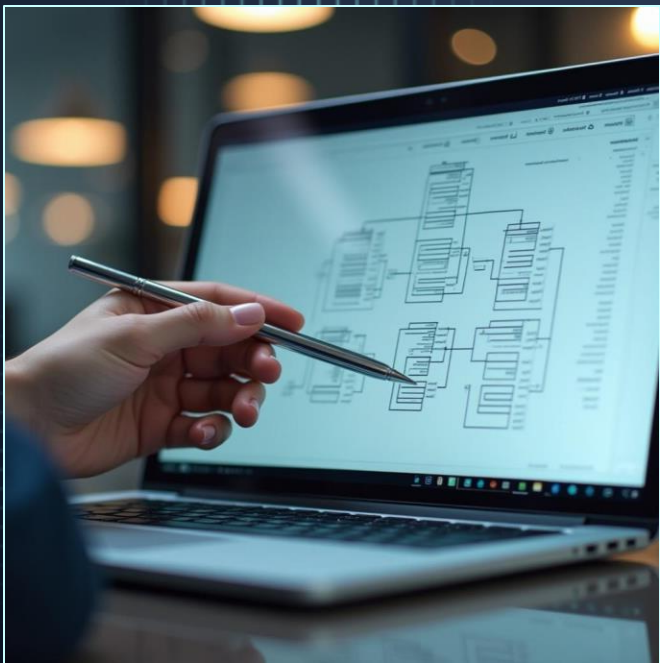
Backend : Node.js + Express

Base de données : MariaDB (SQL)

Sécurité : JWT, bcrypt

Validation : Joi

Documentation : Markdown + Postman



Choix entre SQL et NoSQL

Cette application gère des relations bien définies entre plusieurs entités :

Des utilisateurs (bénévoles et associations)

Des missions créées par les associations

Des candidatures déposées par les bénévoles

Ces données sont **structurées, liées entre elles**, et doivent respecter des **contraintes d'intégrité** (ex : une candidature ne peut exister que si la mission et le bénévole existent). C'est exactement ce pour quoi une base **relationnelle** comme MariaDB est conçue.

```

V VOLUNTEER-PLATFORM
├── > config
├── V controllers
│   ├── applicationController.js
│   ├── missionController.js
│   └── userController.js
├── > database
├── V middlewares
│   ├── authenticateToken.js
│   ├── authorizeRole.js
│   └── validate.js
├── > node_modules
├── V repositories
│   ├── applicationRepository.js
│   ├── missionRepository.js
│   └── userRepository.js
├── V routes
│   ├── applicationRoutes.js
│   ├── missionRoutes.js
│   └── userRoutes.js
├── V services
│   ├── applicationService.js
│   ├── missionService.js
│   └── userService.js
├── V validators
│   ├── applicationValidator.js
│   ├── missionValidator.js
│   └── userValidator.js
├── .env
├── .gitignore
├── app.js
├── package-lock.json
├── package.json
├── PERMISSIONS.md
└── README.md

```

Architecture

Architecture en couches encapsulées avec des classes

- Repository : accès aux données
- Service : logique métier
- Controller : interface HTTP
- Routes : définition des endpoints

Avantages :

Séparation des responsabilités, encapsulation et modularité, testabilité, réutilisabilité, scalabilité et clarté

Structure de la base de données

Réalisé avec Excalidrow

Liaisons et Cardinalités

Un utilisateur (Association) peut créer plusieurs missions

Un utilisateur (Bénévole / volunteer) peut déposer plusieurs candidatures (applications)

Une mission peut recevoir plusieurs candidatures (applications)

Un utilisateur peut avoir plusieurs rôles (si l'on souhaite ajouter un rôle Admin ou autre)

Un rôle peut être attribué à plusieurs utilisateurs
=> table intermédiaire (user_roles)

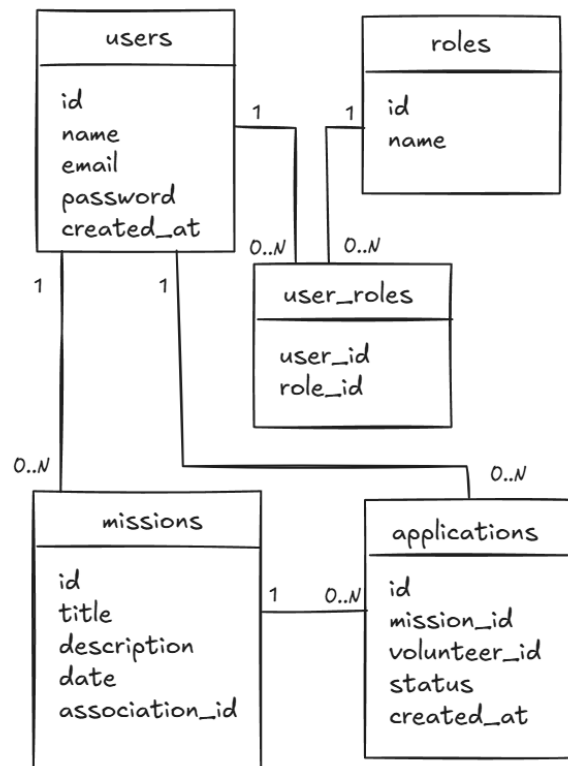
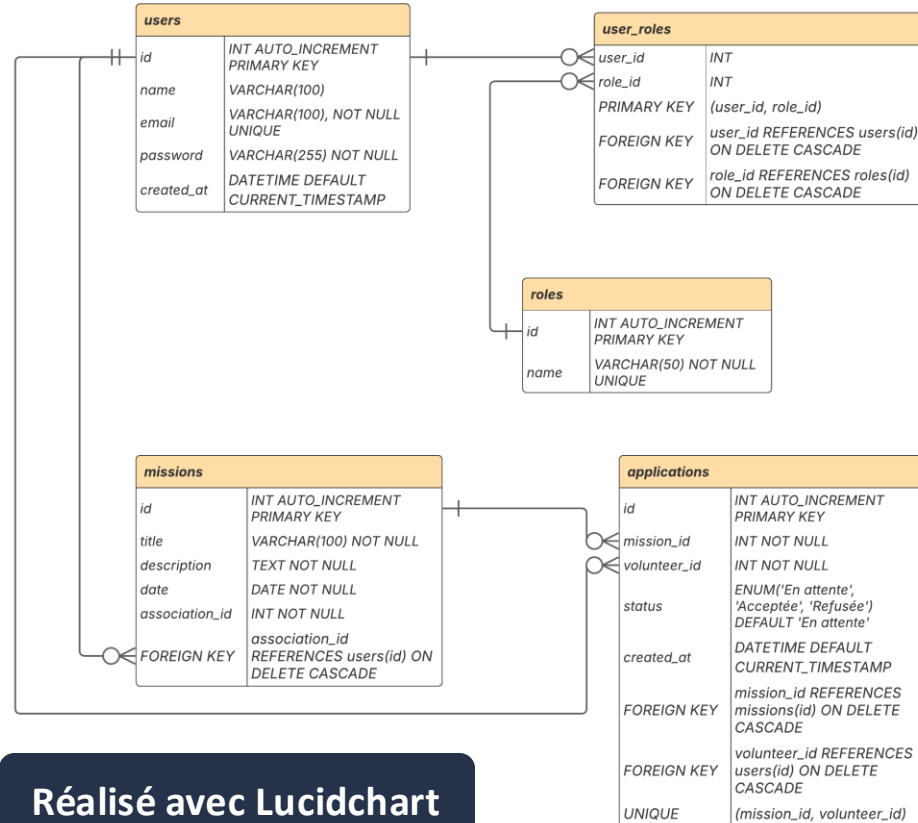


Diagramme Entité-Association volunteer-platform

Philippe Barbosa | September 13, 2025



Réalisé avec Lucidchart

Script SQL

```
database > schema.sql
1 DROP TABLE IF EXISTS applications;
2 DROP TABLE IF EXISTS missions;
3 DROP TABLE IF EXISTS user_roles;
4 DROP TABLE IF EXISTS roles;
5 DROP TABLE IF EXISTS users;
6
7 -- Table des rôles
8 CREATE TABLE roles (
9     id INT AUTO_INCREMENT PRIMARY KEY,
10    name VARCHAR(50) NOT NULL UNIQUE
11 );
12
13 -- Table des utilisateurs
14 CREATE TABLE users (
15     id INT AUTO_INCREMENT PRIMARY KEY,
16     name VARCHAR(100) NOT NULL,
17     email VARCHAR(100) NOT NULL UNIQUE,
18     password VARCHAR(255) NOT NULL,
19     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
20 );
21
22 -- Table intermédiaire
23 CREATE TABLE user_roles (
24     user_id INT NOT NULL,
25     role_id INT NOT NULL,
26     PRIMARY KEY (user_id, role_id),
27     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
28     FOREIGN KEY (role_id) REFERENCES roles(id) ON DELETE CASCADE
29 );
30
31 -- Table des missions
32 CREATE TABLE missions (
33     id INT AUTO_INCREMENT PRIMARY KEY,
34     association_id INT NOT NULL,
35     title VARCHAR(200) NOT NULL,
36     description TEXT NOT NULL,
37     date DATE NOT NULL,
38     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
39     FOREIGN KEY (association_id) REFERENCES users(id) ON DELETE CASCADE
40 );
41
42 -- Table des candidatures
43 CREATE TABLE applications (
44     id INT AUTO_INCREMENT PRIMARY KEY,
45     mission_id INT NOT NULL,
46     volunteer_id INT NOT NULL,
47     status ENUM('En attente', 'Acceptée', 'Refusée') DEFAULT 'En attente',
48     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
49     FOREIGN KEY (mission_id) REFERENCES missions(id) ON DELETE CASCADE,
50     FOREIGN KEY (volunteer_id) REFERENCES users(id) ON DELETE CASCADE,
51     UNIQUE (mission_id, volunteer_id)
52 );
```

```

// Attribution des rôles
await conn.query(`
  INSERT INTO user_roles (user_id, role_id)
  VALUES
    (1, 1),
    (2, 1),
    (3, 2),
    (4, 1),
    (5, 1),
    (6, 2),
    (7, 2);
`);

// Insertion des missions
await conn.query(`
  INSERT INTO missions (title, description, date, association_id)
  VALUES
    ('Collecte alimentaire', 'Aide à la distribution de colis', '2025-09-15', 3),
    ('Nettoyage de plage', 'Sensibilisation et ramassage des déchets', '2025-09-20', 3),
    ('Distribution de vêtements', 'Tri et distribution pour les sans-abris', '2025-09-25', 6),
    ('Atelier numérique', 'Aide aux démarches en ligne pour seniors', '2025-09-30', 7),
    ('Collecte de jouets', 'Organisation d'une collecte pour Noël', '2025-10-05', 6);
`);

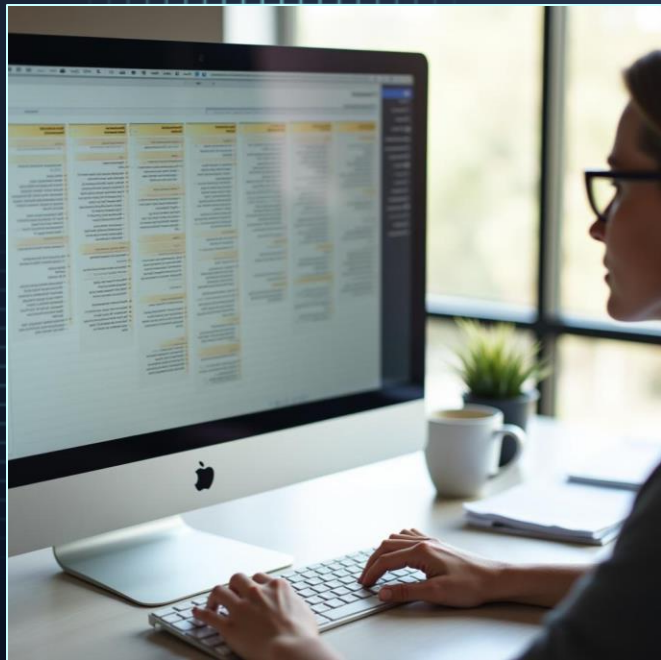
// Insertion des candidatures (applications)
await conn.query(`
  INSERT INTO applications (mission_id, volunteer_id, status)
  VALUES
    (1, 1, 'En attente'),
    (1, 2, 'Acceptée'),
    (2, 1, 'Refusée'),
    (3, 4, 'En attente'),
    (4, 5, 'Acceptée'),
    (5, 1, 'En attente'),
    (5, 2, 'Refusée'),
    (2, 4, 'Acceptée');
`);

```

Jeu de données pour tests

Implémentation automatique d'un
jeu de données avec une fonction seed.js()

Fonctionnalités principales



- Inscription et connexion sécurisée (JWT)
- Rôles utilisateurs : Bénévoles et Associations
- Création de missions par les associations
- Candidature des bénévoles aux missions
- Acceptation ou refus des candidatures
- Filtrage des candidatures par statut
- Validation des données avec Joi
- Documentation API via Postman

Consultez la [documentation des permissions](#) pour comprendre les rôles et accès.

```
import jwt from "jsonwebtoken";

export function authenticateToken(req, res, next) {
  const token = req.cookies?.token;

  if (!token) {
    return res
      .status(401)
      .json({ error: "Authentification requise. Veuillez vous connecter." });
  }

  jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
    if (err) {
      return res
        .status(403)
        .json({
          error: "Authentification invalide. Merci de vous reconnecter.",
        });
    }
    req.user = user;
    next();
  });
}
```

Authentification et rôles

- JWT pour sécuriser les routes avec Middleware `authenticateToken()`
- Middleware `authorizeRole()` pour filtrer l'accès
- Rôles : Bénévoles, Associations

```
export function authorizeRole(requiredRole) {
  return (req, res, next) => {
    if (!req.user || req.user.role !== requiredRole) {
      return res
        .status(403)
        .json({ error: `Accès interdit : réservé aux ${requiredRole}` });
    }
    next();
  };
}
```

missionController

```
import missionService from "../services/missionService.js";

export class MissionController {
  constructor(missionService) {
    this.missionService = missionService;
  }

  async createMission(req, res) {
    try {
      const { title, description, date, association_id } = req.body;
      if (!title || !description || !date || !association_id) {
        return res.status(400).json({ error: "Champs requis manquants" });
      }
      const mission = await this.missionService.createMission({
        title,
        description,
        date,
        association_id,
      });
      res.status(201).json(mission);
    } catch (error) {
      if (error.message === "Association introuvable ou rôle invalide") {
        return res.status(404).json({ error: error.message });
      }
      console.error("Erreur création mission :", error);
      res.status(500).json({ error: "Erreur serveur" });
    }
  }
}
```

Création d'une mission : createMission

Cette méthode vérifie les champs requis, appelle le service métier, et gère les erreurs selon le rôle de l'utilisateur.

```
async apply(req, res) {
  try {
    const { mission_id } = req.body;
    const volunteer_id = req.user.id;

    if (!mission_id || !volunteer_id) {
      return res.status(400).json({ error: "Champs requis manquants" });
    }

    const alreadyApplied = await this.service.hasAlreadyApplied(
      mission_id,
      volunteer_id
    );
    if (alreadyApplied) {
      return res.status(409).json({ error: "Déjà candidat à cette mission" });
    }

    const application = await this.service.apply({
      mission_id,
      volunteer_id,
    });
    res.status(201).json(application);
  } catch (error) {
    if (error.message === "Mission introuvable") {
      return res.status(404).json({ error: "Mission introuvable" });
    }
    if (error.message === "Bénévole introuvable ou rôle invalide") {
      return res.status(404).json({ error: error.message });
    }
    if (error.message === "Association introuvable ou rôle invalide") {
      return res.status(404).json({ error: error.message });
    }

    console.error("Erreur candidature :", error);
    res.status(500).json({ error: "Erreur serveur" });
  }
}
```

Candidature : apply

Cette méthode permet à un bénévole authentifié de postuler à une mission. Elle vérifie que :

- Les champs requis sont présents
- Le bénévole n'a pas déjà postulé
- La mission et les rôles sont valides

Et elle renvoie une réponse HTTP adaptée selon le cas.

applicationController

```
async updateStatus(req, res) {  
  try {  
    const { id } = req.params;  
    const { status } = req.body;  
  
    if (!["Acceptée", "Refusée"].includes(status)) {  
      return res.status(400).json({ error: "Statut invalide" });  
    }  
  
    const updated = await this.service.updateApplicationStatus(id, status);  
    res.status(200).json(updated);  
  } catch (error) {  
    if (error.message === "Candidature introuvable") {  
      return res.status(404).json({ error: "Candidature introuvable" });  
    }  
    if (error.message === "Mission associée introuvable") {  
      return res.status(404).json({ error: "Mission associée introuvable" });  
    }  
    res.status(500).json({ error: "Erreur serveur" });  
  }  
}
```

Mise à jour du statut Acceptation ou refus : updateStatus

La méthode vérifie que le statut fourni est valide ("Acceptée" ou "Refusée"), puis met à jour la candidature correspondante. En cas d'erreur (candidature ou mission introuvable), une réponse adaptée est renvoyée.

applicationRepository

```
async findById(missionId) {
  const conn = await pool.getConnection();
  try {
    const rows = await conn.query(
      `
      SELECT
        m.id AS mission_id,
        a.id AS application_id,
        a.status,
        a.created_at,
        m.title AS mission_title,
        m.date AS mission_date,
        u.name AS volunteer_name,
        u.email AS volunteer_email
      FROM applications a
      JOIN missions m ON a.mission_id = m.id
      JOIN users u ON a.volunteer_id = u.id
      WHERE a.mission_id = ? AND a.status = 'En attente'
      ORDER BY a.created_at ASC
      `,
      [missionId]
    );
    return rows;
  } finally {
    conn.release();
  }
}
```

Affichage des candidatures pour une mission donnée : findById

Cette méthode permet à une association de consulter toutes les candidatures en attente pour une mission donnée. Elle interroge la base de données en joignant les tables applications, missions et users, et retourne les informations pertinentes triées par date de candidature.


```
import Joi from "joi";

export const registerSchema = Joi.object({
  name: Joi.string().min(2).max(100).required().messages({
    "string.empty": "Le nom est requis",
    "string.min": "Le nom doit contenir au moins 2 caractères",
    "string.max": "Le nom ne peut pas dépasser 100 caractères",
  }),
  email: Joi.string().email().required().messages({
    "string.empty": "L'email est requis",
    "string.email": "L'email doit être valide",
  }),
  password: Joi.string()
    .min(6)
    .pattern(new RegExp("(?=.*[A-Z])(?=.*[0-9]).{6,}$"))
    .required()
    .messages({
      "string.empty": "Le mot de passe est requis",
      "string.min": "Le mot de passe doit contenir au moins 6 caractères",
    }),
  role: Joi.string().valid("Bénévoles", "Associations").required().messages({
    "any.only": "Le rôle doit être 'Bénévoles' ou 'Associations'",
    "string.empty": "Le rôle est requis",
  }),
});
```

```
export function validate(schema) {
  return (req, res, next) => {
    const { error } = schema.validate(req.body, { abortEarly: false });
    if (error) {
      const messages = error.details.map((detail) => detail.message);
      return res.status(400).json({ errors: messages });
    }
    next();
  };
}
```

Validation des données avec Joi

Utilisation de la bibliothèque Joi pour sécuriser les données envoyées par l'utilisateur lors de l'inscription. Grâce à un schéma de validation structuré, chaque champ est contrôlé : le nom doit avoir une longueur minimale, l'email doit être valide, le mot de passe doit respecter une certaine complexité, et le rôle doit correspondre à l'un des deux profils autorisés ("Bénévoles" ou "Associations").

Pour appliquer cette validation de manière centralisée et réutilisable, un middleware appelé validate() est utilisé. Ce middleware prend un schéma Joi en paramètre, valide automatiquement le corps de la requête (req.body), et renvoie une erreur 400 Bad Request avec une liste de messages explicites si la validation échoue. Sinon, il passe au middleware suivant avec next().

Documentation & API



Un fichier README présente le projet, les instructions d'installation et la justification technologique :

[Consulter la documentation](#)

La documentation complète de l'API est disponible sur Postman :

[Consulter la documentation](#)

Gestion de projet et suivi des tâches

GITHUB Project a été utilisé pour décomposer le projet
en tâche et le suivi de leur avancement

[Visualiser le projet](#)

Merci !